浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄
学号：3230103913
日期：May 15, 2026
地点：玉泉高分子楼-203

课程名称：人工智能实验　指导老师：魏准　实验类型：设计型实验
实验名称：神经网络与深度学习训练实验　成　绩：　签　名：

装 订 线

一、 实验 6-2：加入 Dropout 的深度神经网络训练

1. 实验目的

本实验在实验 6-1 的深度神经网络基础上引入 dropout，考察随机屏蔽隐藏层神经元后网络的训练表现。网络结构保持为 $25 \to 20 \to 20 \to 20 \to 5$，隐藏层按 Topic 6 Part 1 实验 6-2 的文字要求采用 Sigmoid 激活函数，输出层采用 Softmax 函数，并使用 BP 规则更新权重。

注：课件 slide 16 文字要求使用 Sigmoid 激活函数，而 slide 17 给出的示例代码片段沿用了实验 6-1 中的 ReLU。本实验以 slide 16 的文字要求为准，隐藏层统一采用 Sigmoid 激活函数，仅在隐藏层后插入 dropout（slide 17 演示的关键步骤），输出层 Softmax 不加 dropout。

实验首先比较不加 dropout 与课件示例中的 $p = 0.1$ 设置；为了更清楚地观察 dropout rate 的影响，进一步加入 $p = 0.2$ 和 $p = 0.5$ 两组扩展对照。训练结束后，本文输出各组 Softmax 分类结果，并比较 MSE 随 epoch 的变化曲线。

2. 实验原理

训练数据为 5 个 5×5 数字图案。每个图案展开为 25 维列向量，标签写成 5 类 one-hot 向量。四层权重矩阵的维度为：

$$W_1 \in \mathbb{R}^{20\times 25}, \quad W_2 \in \mathbb{R}^{20\times 20}, \quad W_3 \in \mathbb{R}^{20\times 20}, \quad W_4 \in \mathbb{R}^{5\times 20}.$$

隐藏层的非线性由 Sigmoid 函数给出：

$$\sigma(v) = \frac{1}{1+e^{-v}}, \qquad \sigma'(v) = \sigma(v)(1-\sigma(v)).$$

最后一层使用 Softmax 将输出转换为 5 类概率：

$$\mathrm{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

按照课件中的简化设定，Softmax 层在反向传播时导数直接取 1，因此输出层误差信号可写为：

$$e = d - y, \qquad \delta = e.$$

Dropout 只在训练阶段使用。它随机丢弃一部分隐藏层神经元，并对保留下来的神经元使用 inverted dropout 缩放，使激活值的期望尺度保持不变。设 dropout rate 为 p，则 mask 定义为：

$$m_i = \begin{cases} \frac{1}{1-p}, & \text{第} i \text{ 个神经元被保留} \\ 0, & \text{第} i \text{ 个神经元被丢弃} \end{cases}$$

于是隐藏层传给下一层的实际输出为：

$$\tilde{y}_l = y_l \odot m_l.$$

本实验只对三个隐藏层施加 dropout，输出层不加 dropout；训练完成后的验证和结果输出均采用普通前向传播，不再随机屏蔽神经元。由于前向传播中被屏蔽的神经元不参与下一层计算，反向传播时也需要用同一个 mask 过滤对应梯度。以第三隐藏层为例：

$$\delta_3 = \sigma'(v_3) \odot m_3 \odot (W_4^T \delta).$$

第二、第一隐藏层的误差信号依次为：

$$\delta_2 = \sigma'(v_2) \odot m_2 \odot (W_3^T \delta_3), \qquad \delta_1 = \sigma'(v_1) \odot m_1 \odot (W_2^T \delta_2).$$

对应的 SGD-BP 权重更新为：

$$W_4 \leftarrow W_4 + \alpha \delta \tilde{y}_3^T, \quad W_3 \leftarrow W_3 + \alpha \delta_3 \tilde{y}_2^T,$$

$$W_2 \leftarrow W_2 + \alpha \delta_2 \tilde{y}_1^T, \quad W_1 \leftarrow W_1 + \alpha \delta_1 x^T.$$

3. 代码实现

Listing 1 给出了 inverted dropout mask 的实现。函数先根据保留比例 $1-p$ 计算需要保留的神经元数量，再随机选择下标，并把这些位置设为 $1/(1-p)$。代码中使用 `ym.flat[idx]` 写入 mask，这样无论激活值是普通向量还是 $(20, 1)$ 形式的列向量，下标都会按展平后的位置解释，避免直接写 `ym[idx]` 时可能出现的行索引问题。

Listing 1: Dropout.py：Dropout mask 生成函数

```python
import numpy as np


def Dropout(y, ratio):
    """Inverted dropout mask used only during training."""
    if ratio < 0.0 or ratio >= 1.0:
        raise ValueError("dropout ratio must be in [0, 1)")

    ym = np.zeros_like(y)
    num = round(y.size * (1.0 - ratio))
    idx = np.random.choice(y.size, num, replace=False)
    ym.flat[idx] = 1.0 / (1.0 - ratio)
    return ym
```

Listing 2 是一轮带 dropout 的 SGD-BP 训练。三个隐藏层先计算 Sigmoid 激活，再分别乘以 dropout mask 得到 `y1_drop`、`y2_drop` 和 `y3_drop`；输出层仍直接接 Softmax，不再额外使用 dropout。反向传播时，各隐藏层 delta 同步乘入前向传播使用过的 mask，使被屏蔽神经元在本次更新中不产生梯度贡献。

Listing 2: DeepDropout.py：Sigmoid 隐藏层与 Dropout 训练函数

```python
import numpy as np

from Dropout import Dropout
from Sigmoid import Sigmoid, Sigmoid_derivative
from Softmax import Softmax


def DeepDropout(W1, W2, W3, W4, X, D, alpha=0.02, ratio=0.1):
    """One SGD epoch for a 25-20-20-20-5 sigmoid network with dropout."""
    n_samples = X.shape[2]
```

装 订 线

```
    order = np.random.permutation(n_samples)
    for k in order:        x = np.reshape(X[:, :, k], (25, 1))

        v1 = W1 @ x
        y1 = Sigmoid(v1)
        m1 = Dropout(y1, ratio)
        y1_drop = y1 * m1

        v2 = W2 @ y1_drop
        y2 = Sigmoid(v2)
        m2 = Dropout(y2, ratio)
        y2_drop = y2 * m2

        v3 = W3 @ y2_drop
        y3 = Sigmoid(v3)
        m3 = Dropout(y3, ratio)
        y3_drop = y3 * m3

        v = W4 @ y3_drop
        y = Softmax(v)

        d = D[k, :].reshape(5, 1)
        e = d - y
        delta = e

        e3 = W4.T @ delta
        delta3 = Sigmoid_derivative(v3) * m3 * e3
        e2 = W3.T @ delta3
        delta2 = Sigmoid_derivative(v2) * m2 * e2
        e1 = W2.T @ delta2
        delta1 = Sigmoid_derivative(v1) * m1 * e1

        dW4 = alpha * delta @ y3_drop.T
        dW3 = alpha * delta3 @ y2_drop.T
        dW2 = alpha * delta2 @ y1_drop.T
        dW1 = alpha * delta1 @ x.T

        W4 = W4 + dW4
        W3 = W3 + dW3
        W2 = W2 + dW2
        W1 = W1 + dW1

    return W1, W2, W3, W4
```

Listing 3 和 Listing 4 给出了统一的训练入口。所有实验组使用相同的初始权重、学习率、训练轮数和训练数据；每个 epoch 结束后，均切换到不使用 dropout 的推理模式计算训练集 MSE。循环中最后一次前向传播得到的输出直接作为最终输出返回，避免在权重不再变化时重复计算。这样记录到的误差反映的是模型本身的输出，而不是某一次随机 mask 下的临时输出。

Listing 3: exp6_2.py：训练函数与 MSE 记录

```
def train_sigmoid(dropout=False, seed=7, epochs=5000, alpha=0.02, ratio=0.1):
    X, D = make_dataset()
    W1, W2, W3, W4 = initialize_weights(seed)

    np.random.seed(seed + 100)
    history = []
    for _ in range(epochs):
        if dropout:
            W1, W2, W3, W4 = DeepDropout(W1, W2, W3, W4, X, D, alpha=alpha, ratio=ratio)
        else:
            W1, W2, W3, W4 = DeepSigmoid(W1, W2, W3, W4, X, D, alpha=alpha)

        y = predict_sigmoid(W1, W2, W3, W4, X)
        history.append(mse(y, D))

    return X, D, W1, W2, W3, W4, np.array(history), y
```

Listing 4: exp6_2.py：实验参数、dropout rate 对照与结果输出

装 订 线

```python
seed = 7
epochs = 5000
alpha = 0.02
ratios = [0.1, 0.2, 0.5]
fig_dir = Path(__file__).resolve().parents[2] / "Lab9实验报告" / "figures"

X, D, W1, W2, W3, W4, no_dropout_history, no_dropout_y = train_sigmoid(
    dropout=False, seed=seed, epochs=epochs, alpha=alpha
)

results = [
    {
        "name": "No dropout",
        "history": no_dropout_history,
        "y": no_dropout_y,
        "color": "darkorange",
    }
]

dropout_weights = None
colors = ["steelblue", "seagreen", "crimson"]
for ratio, color in zip(ratios, colors):
    _, _, Wd1, Wd2, Wd3, Wd4, dropout_history, dropout_y = train_sigmoid(
        dropout=True, seed=seed, epochs=epochs, alpha=alpha, ratio=ratio
    )
    if ratio == 0.1:
        dropout_weights = (Wd1, Wd2, Wd3, Wd4)
    results.append(
        {
            "name": f"Dropout p={ratio}",
            "history": dropout_history,
            "y": dropout_y,
            "color": color,
        }
    )

save_figures(fig_dir, results)

if dropout_weights is not None:
    W1, W2, W3, W4 = dropout_weights

print("Experiment 6-2: Sigmoid + Softmax network with dropout")
print(f"seed={seed}, alpha={alpha}, epochs={epochs}, dropout ratios={ratios}")
print(f"W1={W1.shape}, W2={W2.shape}, W3={W3.shape}, W4={W4.shape}")
print()
for result in results:
    print_outputs(result["name"], result["history"], result["y"], D)
print(f"figures saved to {fig_dir}")
```

4. 实验结果

实验统一采用随机种子 7、学习率 $\alpha = 0.02$，训练 5000 epochs。除不加 dropout 的基线外，本文分别测试 $p = 0.1$、$p = 0.2$ 和 $p = 0.5$ 三种 dropout rate。各组从同一组初始权重出发，权重均由 $[-1, 1]$ 上的均匀分布采样得到。运行 `python Code/EXP6/exp6_2.py` 后，最终结果如表 1 所示。

训练方式	最终 MSE	准确率	样本 1	样本 2	样本 3	样本 4	样本 5
不加 dropout	3.291×10^{-6}	100%	0.997276	0.995416	0.996624	0.997040	0.996402
$p = 0.1$	1.436×10^{-7}	100%	0.999555	0.998972	0.999534	0.999132	0.999419
$p = 0.2$	8.72×10^{-8}	100%	0.999783	0.999313	0.999720	0.999477	0.999241
$p = 0.5$	8.064×10^{-2}	60%	0.995875	0.287483	0.411194	0.993037	0.306158

Table 1: 实验 6-2 不同 dropout rate 下的最终训练结果。样本 1–样本 5 列表示对应样本在正确类别上的 Softmax 概率

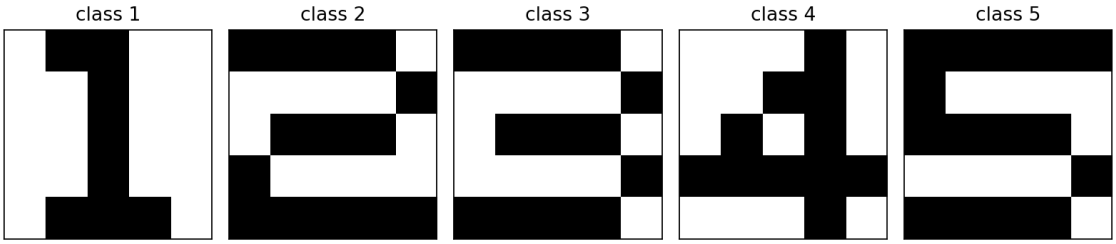


Figure 1: 实验 6-2 训练使用的五个 5×5 数字图案

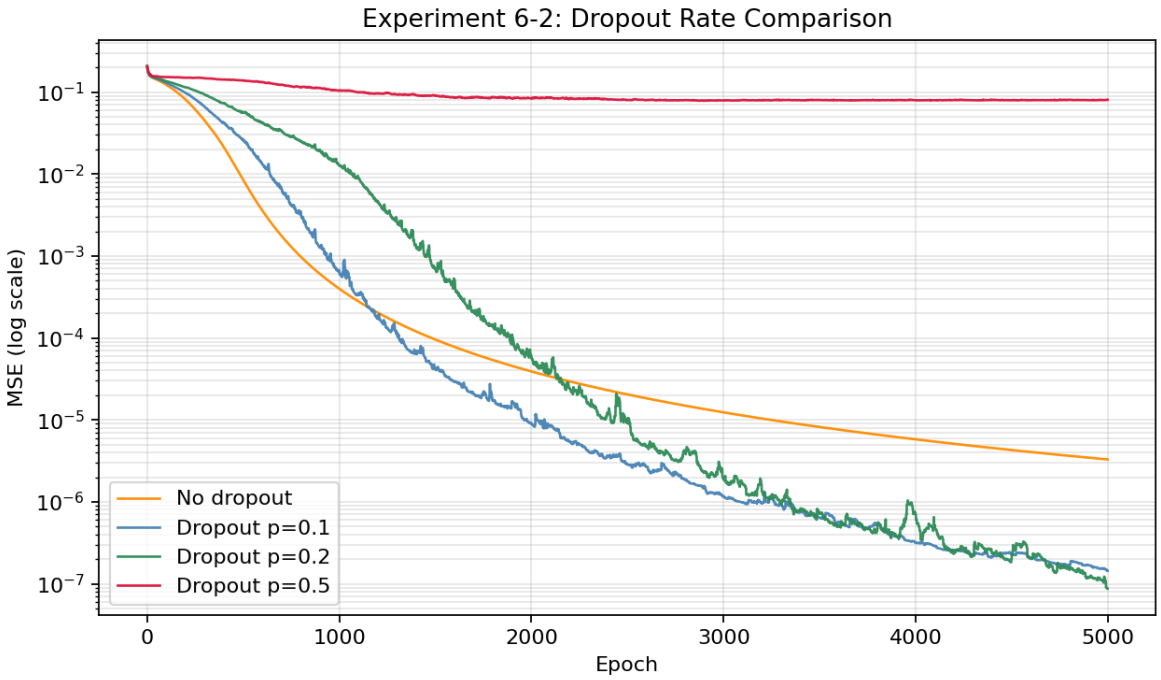


Figure 2: 实验 6-2：不同 dropout rate 下的 MSE vs Epoch 曲线

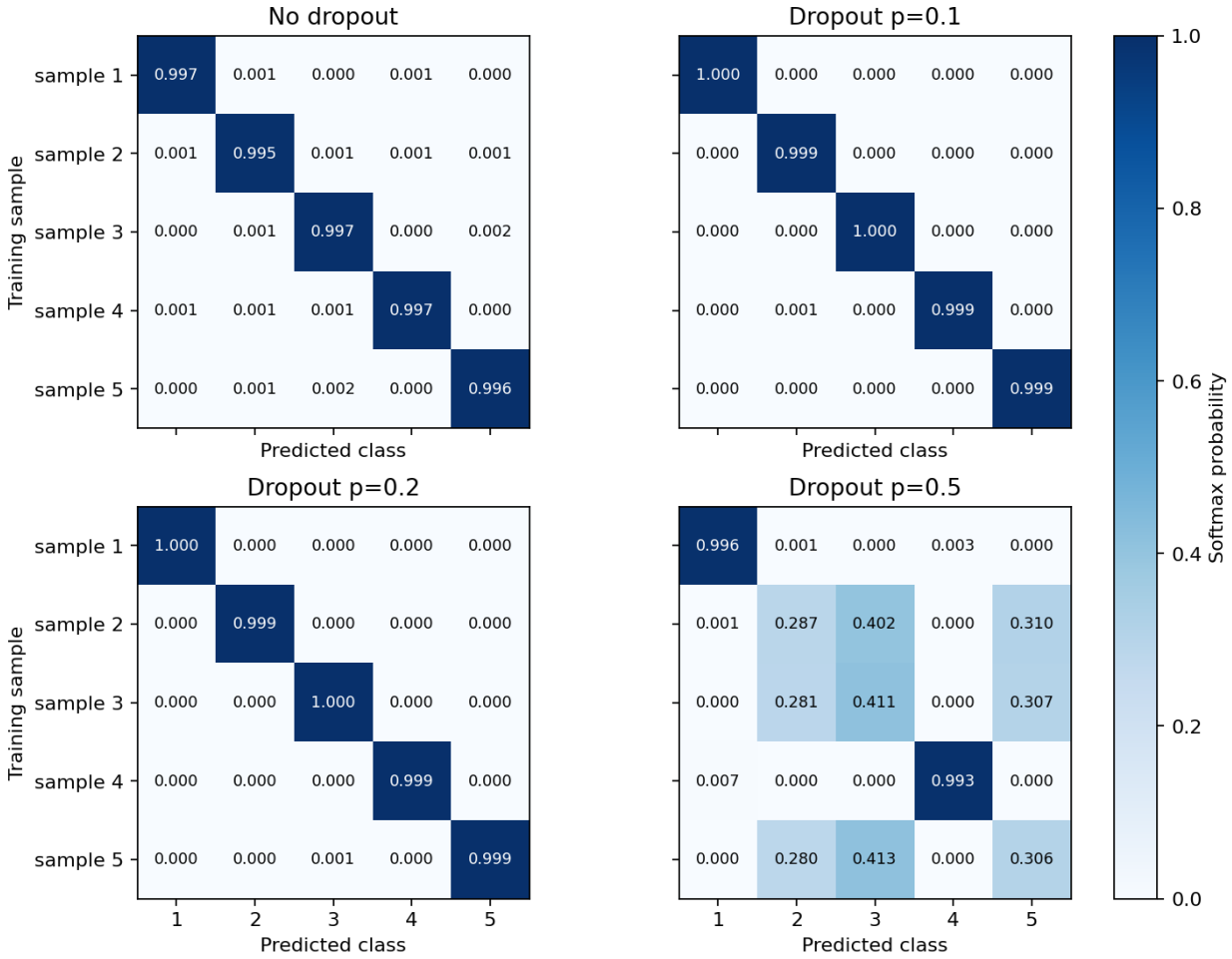


Figure 3: 实验 6-2：不同 dropout rate 下的 Softmax 输出概率矩阵

5. 结果分析

表 1 显示，不加 dropout、$p = 0.1$ 和 $p = 0.2$ 三组都能把 5 个训练样本全部分对，说明在这些设置下网络已经学到数字图案与 one-hot 标签之间的对应关系。就本次随机种子和训练参数而言，$p = 0.1$ 与 $p = 0.2$ 的训练集 MSE 分别降至 1.436×10^{-7} 和 8.72×10^{-8}，低于不加 dropout 的 3.291×10^{-6}。不过，这里只有 5 个训练样本，没有独立验证集或测试集，因此这些数值只能说明当前训练集上的优化结果，不能直接作为 dropout 提升泛化能力的证据。

从图 2 可以进一步观察不同 dropout rate 对训练过程的影响。不加 dropout 时，MSE 曲线最平滑；$p = 0.1$ 和 $p = 0.2$ 因为每轮训练都引入随机 mask，曲线局部有更明显的起伏，但整体仍持续下降。$p = 0.2$ 的扰动比 $p = 0.1$ 更强，在本次运行中最终训练集 MSE 略低。与这两组不同，$p = 0.5$ 的曲线长期停留在较高误差水平，最终准确率只有 60%。这说明在本实验这种样本极少、每层仅 20 个隐藏单元的小网络中，一次丢弃一半隐藏神经元会明显增加优化难度，网络难以稳定记住所有样本。

图 3 展示了最终 Softmax 概率矩阵。不加 dropout、$p = 0.1$ 和 $p = 0.2$ 的最大概率均集中在对角线上，其中 $p = 0.1$ 与 $p = 0.2$ 的非目标类别概率更接近 0，输出分布更尖锐。$p = 0.5$ 则出现明显退化：样本 2 和样本 5 被推向第 3 类附近，样本 3 虽然仍被判为第 3 类，但正确类别概率也只有约 0.411。由此可见，dropout rate 不是越大越好，它需要与网络规模、训练样本量和训练轮数匹配；在当前设置下，过强 dropout 反而阻碍收敛。

6. 实验小结

本实验在 Sigmoid 深度网络中加入 dropout，并比较了不加 dropout、$p = 0.1$、$p = 0.2$ 与 $p = 0.5$ 四种训练设置。结果表明，$p = 0.1$ 和 $p = 0.2$ 都能稳定收敛，并在训练集上达到 100% 准确率；$p = 0.5$ 由于随机丢弃过强，最终只达到 60%。因此，在当前小样本任务中，适度 dropout 可以作为训练过程中的随机扰动，但过大的 dropout rate 会明显削弱网络的有效表达与优化能力。由于本实验没有独立测试集，训练集 MSE 的高低主要反映当前优化路径，不能单独用来证明 dropout 具有更强泛化能力。

装 订 线

装 订 线

二、 实验 7-1：卷积神经网络训练与测试

1. 实验目的

本实验根据 Topic 7 Part 1 中实验 7-1 的要求，使用给定的卷积层 `Conv`、池化层 `Pool` 和 `LoadMnistData` 数据读取函数，结合 mini-batch 与 momentum 规则搭建并训练一个类似 LeNet 的卷积神经网络。实验数据从 `Code/EXP7/References/MNIST` 中的 MNIST 文件读取，取前 2000 张图像作为训练集，再取之后 300 张图像作为测试集，二者不重合。训练完成后输出预测结果的准确度，即

$$\text{Accuracy} = \frac{\text{预测正确数量}}{N}, \quad N = 300.$$

2. 实验原理

给出的卷积神经网络结构如表 2 所示。输入为 28×28 灰度图像；卷积层使用 20 个 9×9 卷积核，步长 $S = 1$、补零 $P = 0$，因此卷积输出大小为 $20 \times 20 \times 20$；之后经过 ReLU 与 2×2 平均池化，得到 $10 \times 10 \times 20$ 的特征图。池化结果展开为 2000 维向量，送入 100 节点全连接隐藏层，最后经 10 节点 Softmax 层输出数字 0–9 的分类概率。

层	参数说明	激活函数
Input	28×28 nodes	-
Convolution	20 convolution filters (9×9), $P = 0, S = 1$	ReLU
Pooling	1 mean pooling (2×2)	-
Hidden	100 nodes	ReLU
Output	10 nodes	Softmax

Table 2: 实验 7-1 卷积神经网络结构

卷积层输出尺寸满足：

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \qquad H_2 = \frac{H_1 - F + 2P}{S} + 1.$$

代入 $W_1 = H_1 = 28$、$F = 9$、$P = 0$、$S = 1$，得到 $W_2 = H_2 = 20$。池化层使用 2×2 平均池化并以 2 为步长下采样，因此 20×20 特征图变为 10×10。

输出层仍采用课件中的简化处理，Softmax 反向传播中令

$$\delta = d - y,$$

其中 d 是由标签转换得到的 one-hot 向量。全连接隐藏层误差信号为：

$$\delta_5 = (y_5 > 0) \odot (W_o^T \delta).$$

池化层的反向传播将 10×10 的误差按平均池化权重均匀分配回 2×2 区域：

$$e_2 = \text{kron}(e_3, \mathbf{1}_{2\times 2})/4.$$

卷积层梯度由输入图像与卷积层误差做 valid 卷积得到。mini-batch 中先累积各样本梯度，再取平均；momentum 更新形式为：

$$m_t = \alpha \overline{\Delta W_t} + \beta m_{t-1}, \qquad W_t = W_{t-1} + m_t.$$

装 订

3. 代码实现

Listing 5 和 Listing 6 分别给出了卷积层与平均池化层的实现。卷积层中根据 `signal.convolve2d` 的定义先将卷积核旋转 180°，再做 valid 卷积；池化层使用 2×2 平均模板并每隔 2 个像素取一次结果。

Listing 5: Conv.py：卷积层定义

```python
import numpy as np
from scipy import signal


def Conv(x, W):
    (wrow, wcol, numFilters) = W.shape
    (xrow, xcol) = x.shape

    yrow = xrow - wrow + 1
    ycol = xcol - wcol + 1

    y = np.zeros((yrow, ycol, numFilters))

    for k in range(numFilters):
        filt = W[:, :, k]
        filt = np.rot90(np.squeeze(filt), 2)
        y[:, :, k] = signal.convolve2d(x, filt, "valid")

    return y
```

Listing 6: Pool.py：平均池化层定义

```python
import numpy as np
from scipy import signal


def Pool(x):
    (xrow, xcol, numFilters) = x.shape
    y = np.zeros((int(xrow / 2), int(xcol / 2), numFilters))

    for k in range(numFilters):
        filt = np.ones((2, 2)) / (2 * 2)
        image = signal.convolve2d(x[:, :, k], filt, "valid")
        y[:, :, k] = image[::2, ::2]

    return y
```

Listing 7 给出了 `LoadMnistData` 的数据读取实现。函数直接读取 `References/MNIST` 中的 IDX 格式 MNIST 图像与标签文件，支持 `.gz` 压缩文件；读取时先检查文件是否存在，再校验 image 和 label 文件的 magic number，最后返回形状为 $(N, 28, 28)$ 的图像数组和 $(N, 1)$ 的标签数组。

Listing 7: LoadMnistData.py：MNIST 数据读取函数

```python
def _open_idx(path):
    if path.suffix == ".gz":
        return gzip.open(path, "rb")
    return open(path, "rb")


def LoadMnistData(imagefile, labelfile):
    imagefile = Path(imagefile)
    labelfile = Path(labelfile)
    if not imagefile.exists():
        raise FileNotFoundError(f"MNIST image file not found: {imagefile}")
    if not labelfile.exists():
        raise FileNotFoundError(f"MNIST label file not found: {labelfile}")

    with _open_idx(imagefile) as images:
        header = images.read(16)
        magic, number_of_images, rows, cols = struct.unpack(">IIII", header)
        if magic != 2051:
```

```
            raise ValueError(f"invalid MNIST image file: {imagefile}")
        pixel_count = number_of_images * rows * cols        data = np.frombuffer(images.read(pixel_count), dtype=np.uint8)
        x = data.reshape(number_of_images, rows, cols).astype(np.float32)

    with _open_idx(labelfile) as labels:
        magic, number_of_labels = struct.unpack(">II", labels.read(8))
        if magic != 2049:
            raise ValueError(f"invalid MNIST label file: {labelfile}")
        y = np.frombuffer(labels.read(number_of_labels), dtype=np.uint8)
        y = y.reshape(-1, 1)

    if number_of_images != number_of_labels:
        raise ValueError("number of labels did not match the number of images")

    return x, y
```

为了避免长代码框跨页挤压，训练函数拆成两段展示。Listing 8 给出了 mini-batch 中单个样本的前向传播、one-hot 标签构造和反向传播；Listing 9 给出了梯度平均、momentum 更新、每个 epoch 后的训练/测试准确率记录和函数返回值。

Listing 8: MnistConv.py：CNN 前向传播与反向传播核心

```
def MnistConv(
    W1,
    W5,
    Wo,
    X,
    D,
    epochs=10,
    alpha=0.01,
    beta=0.95,
    bsize=10,
    seed=123,
    X_test=None,
    D_test=None,
):
    momentum1 = np.zeros_like(W1)
    momentum5 = np.zeros_like(W5)
    momentumo = np.zeros_like(Wo)

    N = len(D)
    rng = np.random.default_rng(seed)
    train_history = []
    test_history = []

    for _ in range(epochs):
        order = rng.permutation(N)

        for begin in range(0, N, bsize):
            batch_idx = order[begin : begin + bsize]
            dW1 = np.zeros_like(W1)
            dW5 = np.zeros_like(W5)
            dWo = np.zeros_like(Wo)

            for k in batch_idx:
                x = X[k, :, :]
                y1 = Conv(x, W1)
                y2 = ReLU(y1)
                y3 = Pool(y2)
                y4 = np.reshape(y3, (-1, 1))
                v5 = np.matmul(W5, y4)
                y5 = ReLU(v5)
                v = np.matmul(Wo, y5)
                y = Softmax(v)

                d = np.zeros((10, 1))
                d[int(D[k][0])][0] = 1.0

                e = d - y
                delta = e
                e5 = np.matmul(Wo.T, delta)
```

```
                delta5 = (y5 > 0) * e5
                e4 = np.matmul(W5.T, delta5)                e3 = np.reshape(e4, y3.shape)

                e2 = np.repeat(np.repeat(e3, 2, axis=0), 2, axis=1) / 4.0
                delta2 = (y2 > 0) * e2

                delta1_x = np.zeros_like(W1)
                for c in range(W1.shape[2]):
                    delta1_x[:, :, c] = signal.convolve2d(
                        x,
                        np.rot90(delta2[:, :, c], 2),
                        "valid",
                    )

                dW1 = dW1 + delta1_x
                dW5 = dW5 + np.matmul(delta5, y4.T)
                dWo = dWo + np.matmul(delta, y5.T)
```

Listing 9: MnistConv.py：mini-batch 平均、momentum 更新与准确率记录

```
            scale = float(len(batch_idx))
            dW1 = dW1 / scale
            dW5 = dW5 / scale
            dWo = dWo / scale

            momentum1 = alpha * dW1 + beta * momentum1
            W1 = W1 + momentum1

            momentum5 = alpha * dW5 + beta * momentum5
            W5 = W5 + momentum5

            momentumo = alpha * dWo + beta * momentumo
            Wo = Wo + momentumo

        train_pred, _ = Predict(W1, W5, Wo, X)
        train_history.append(float(np.mean(train_pred == D.reshape(-1))))

        if X_test is not None and D_test is not None:
            test_pred, _ = Predict(W1, W5, Wo, X_test)
            test_history.append(float(np.mean(test_pred == D_test.reshape(-1))))

    return W1, W5, Wo, np.array(train_history), np.array(test_history)
```

Listing 10 和 Listing 11 给出了实验入口。主程序初始化三组权重，从 `References/MNIST` 中划分训练集和测试集，并把测试集传入训练函数，用于记录每个 epoch 后的测试准确率。

Listing 10: exp7_1.py：权重初始化与数据划分

```
def initialize_weights(seed=7):
    rng = np.random.default_rng(seed)
    W1 = rng.normal(0.0, np.sqrt(2.0 / (9 * 9)), (9, 9, 20))
    W5 = rng.normal(0.0, np.sqrt(2.0 / 2000), (100, 2000))
    Wo = rng.normal(0.0, np.sqrt(2.0 / 100), (10, 100))
    return W1, W5, Wo


def load_experiment_data():
    data_dir = Path(__file__).resolve().parent / "References" / "MNIST"
    images, labels = LoadMnistData(
        data_dir / "t10k-images-idx3-ubyte.gz",
        data_dir / "t10k-labels-idx1-ubyte.gz",
    )
    if images.shape[0] < 2300 or labels.shape[0] < 2300:
        raise ValueError("experiment 7-1 needs at least 2300 MNIST samples")

    images = np.divide(images, 255.0)
    X = images[0:2000, :, :]
    D = labels[0:2000]
    X_test = images[2000:2300, :, :]
    D_test = labels[2000:2300]
```

```
    return X, D, X_test, D_test
```

Listing 11: exp7_1.py：训练、测试与图像保存入口

```
        ax.set_title(f"#{idx} T:{true_label} P:{pred_label}\nC:{confidence:.2f}", fontsize=8)
        ax.set_xticks([])
        ax.set_yticks([])
    plt.suptitle(f"Experiment 7-1: Misclassified Samples ({len(wrong)})")
    plt.tight_layout()
    plt.savefig(fig_dir / "Figure_exp7_1_errors.png", dpi=160)
    plt.close()


def main():
    seed = 7
    epochs = 10
    alpha = 0.01
    beta = 0.95
    bsize = 10
    fig_dir = Path(__file__).resolve().parents[2] / "Lab9实验报告" / "figures"

    X, D, X_test, D_test = load_experiment_data()
    W1, W5, Wo = initialize_weights(seed)
    W1, W5, Wo, train_history, test_history = MnistConv(
        W1,
        W5,
        Wo,
        X,
        D,
        epochs=epochs,
        alpha=alpha,
        beta=beta,
        bsize=bsize,
        X_test=X_test,
        D_test=D_test,
    )

    pred_train, _ = Predict(W1, W5, Wo, X)
    pred_test, probs_test = Predict(W1, W5, Wo, X_test)
    train_acc = float(np.mean(pred_train == D.reshape(-1)))
    test_correct = int(np.sum(pred_test == D_test.reshape(-1)))
    test_acc = test_correct / len(D_test)
    conf = confusion_matrix(D_test, pred_test)
    errors = error_summary(D_test, pred_test)

    save_figures(
        fig_dir,
```

4. 实验结果

实验参数为：随机种子 7，训练轮数 10，学习率 $\alpha = 0.01$，momentum 系数 $\beta = 0.95$，mini-batch 大小为 10。权重初始化采用 He 初始化，其中 $W_1 \in \mathbb{R}^{9\times9\times20}$，$W_5 \in \mathbb{R}^{100\times2000}$，$W_o \in \mathbb{R}^{10\times100}$。运行 `python Code/EXP7/exp7_1.py` 后得到结果如表 3 所示。

训练样本数	测试样本数	Epochs	训练准确率	测试正确数	测试准确率
2000	300	10	100.00%	283 / 300	94.33%

Table 3: 实验 7-1 CNN 训练与测试结果

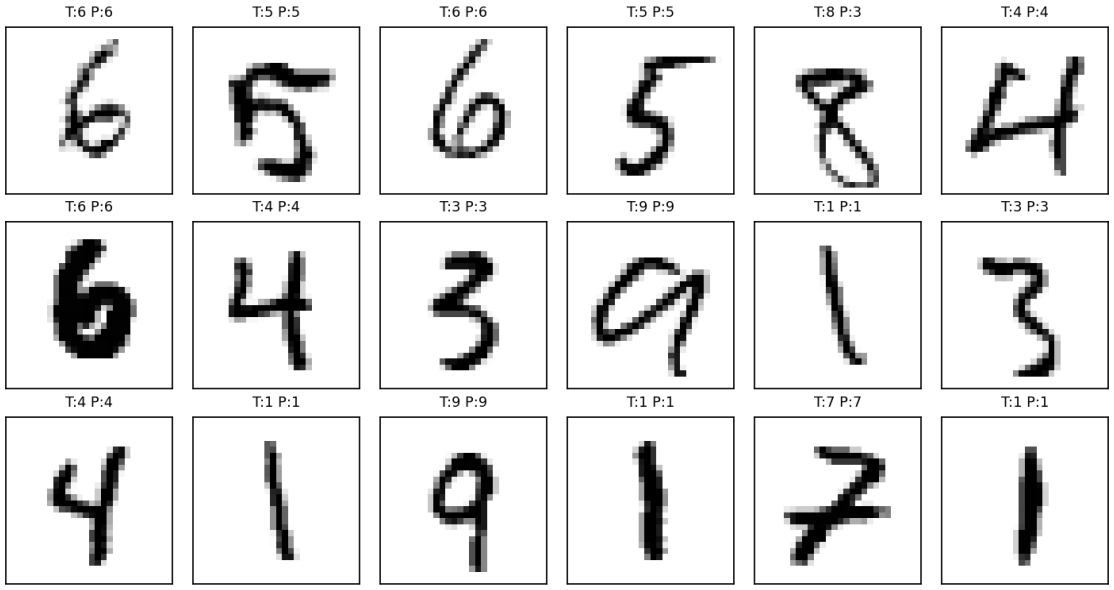


Figure 4: 实验 7-1：部分测试样本及预测结果，T 表示真实标签，P 表示预测标签



Figure 5: 实验 7-1：mini-batch momentum 训练过程中的训练/测试错误率（对数坐标）

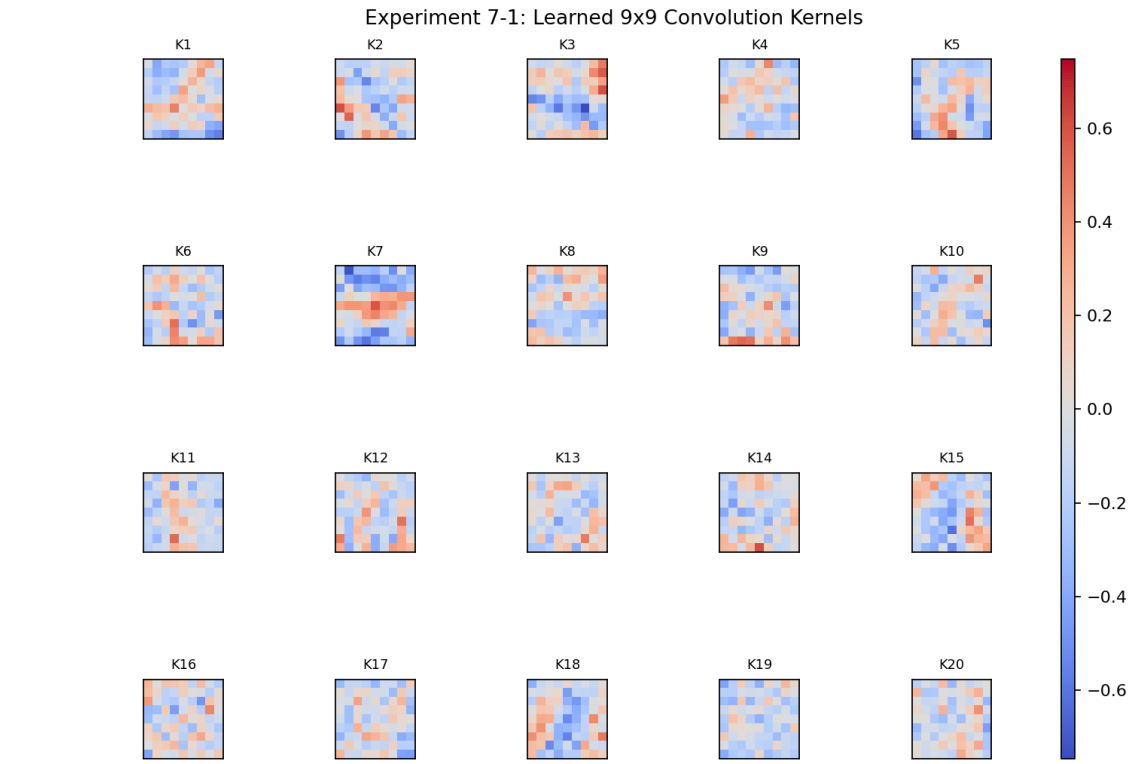


Figure 6: 实验 7-1：训练后的 20 个 9×9 卷积核

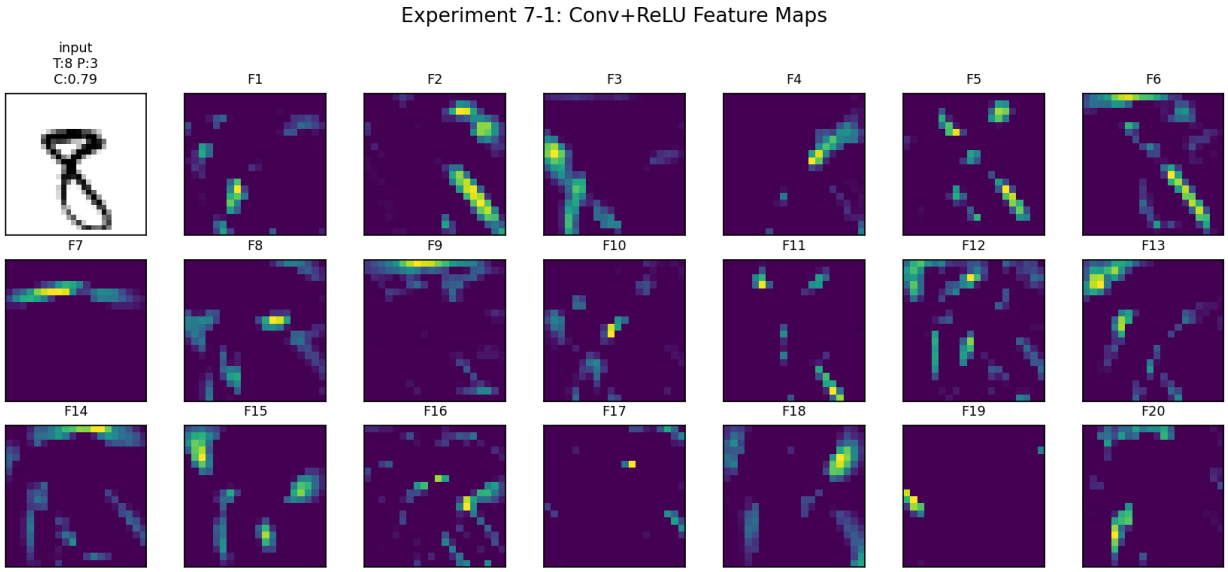


Figure 7: 实验 7-1：错分样本经过 Conv+ReLU 后的 20 张 feature map

装 订 线

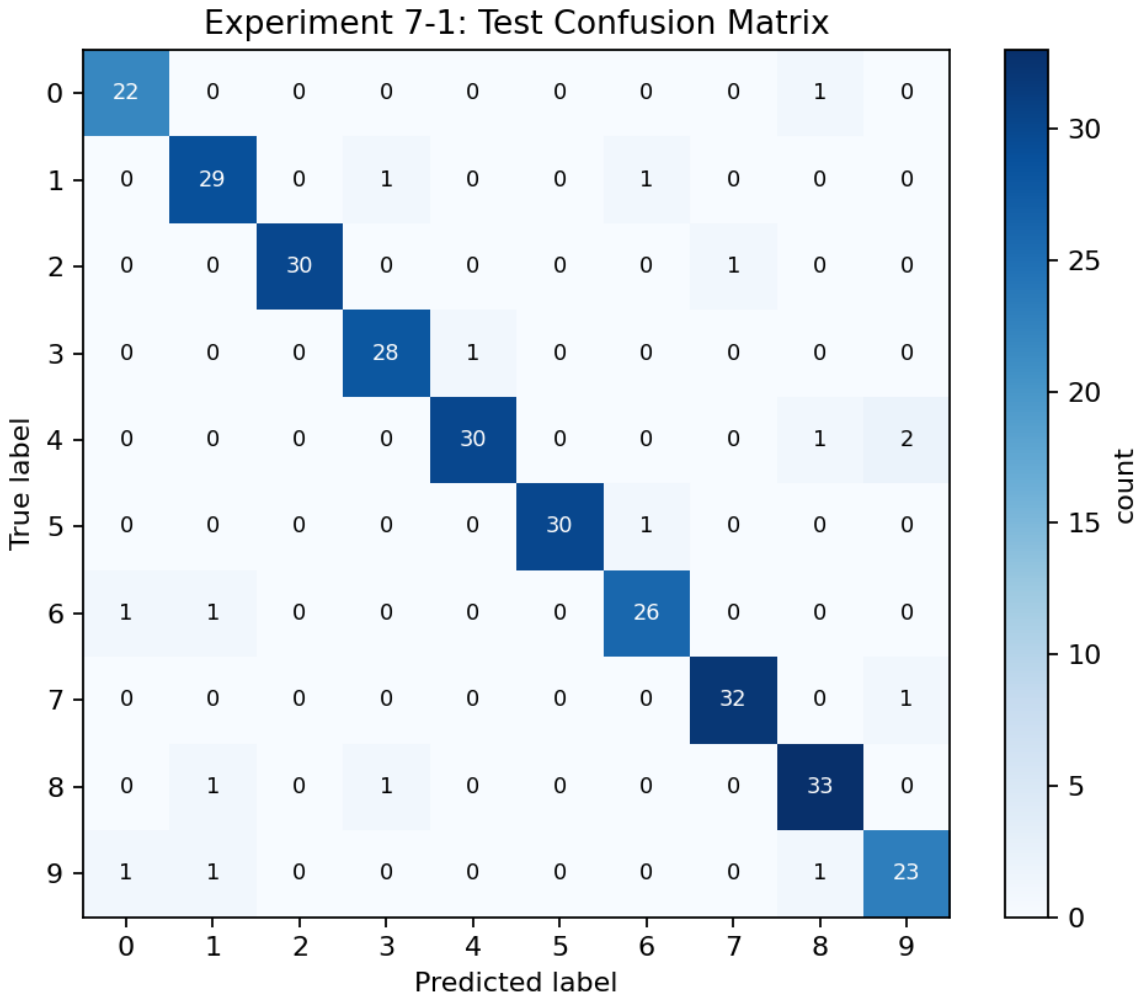


Figure 8: 实验 7-1：300 张测试图像的预测混淆矩阵

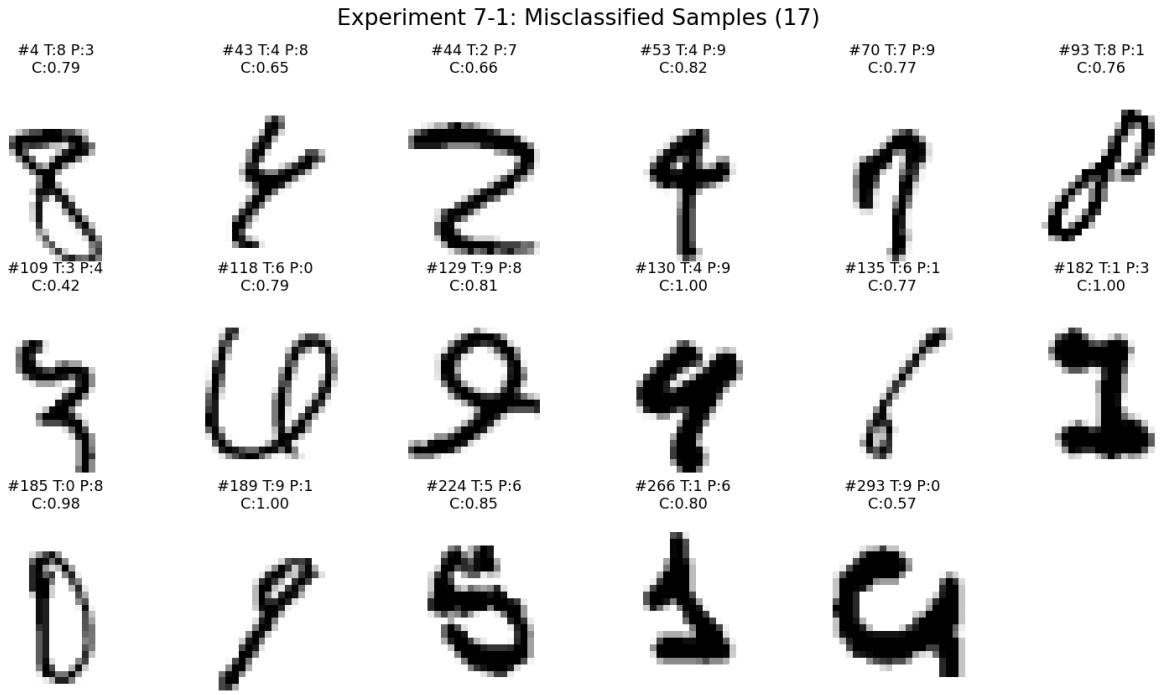


Figure 9: 实验 7-1：全部错分测试样本，C 表示预测类别的 Softmax 置信度

5. 结果分析

图 5 用错误率的对数坐标展示训练过程，因此高准确率区间内的变化不会被压缩在图像顶部。由于对数坐标不能显示 0，第 10 个 epoch 的训练错误率在图中按 10^{-3} 下限显示，实际训练准确率仍为 100.00%。训练准确率由第 1 个 epoch 的 89.75% 上升到第 10 个 epoch 的 100.00%，测试准确率由 86.33% 上升到 94.33%。第 5 个 epoch 时测试准确率为 92.67%，之后仍有提升，说明 5 个 epoch 还不足以充分训练该网络。第 8 到第 10 个 epoch 的测试准确率在 93%–94.33% 附近波动，而训练准确率继续逼近 100%，表明模型已经开始记住训练集，但测试集性能尚未出现明显下降。

图 6 展示了训练后的 20 个卷积核。部分卷积核呈现出方向性边缘、短笔画和局部明暗对比的结构，这说明卷积层不只是记忆像素位置，而是在学习能响应数字笔画的局部模式。图 7 选择了一个被识别为 3 的 8 类样本作为例子，可以看到不同 feature map 对输入图像的不同区域产生响应：有的图强调上半部分横向笔画，有的图强调下半部分弯曲结构。这些中间表示解释了卷积层如何把原始像素转换为后续全连接层可利用的特征。

最终测试集准确率为 $283/300 = 94.33\%$，共有 17 张测试图像被错分。由图 8 和图 9 可见，错误主要发生在笔画形态相近或书写不规范的样本上。错误类型中 $4 \to 9$ 出现 2 次，其余错误类型各出现 1 次，包括 $0 \to 8$、$1 \to 3$、$1 \to 6$、$2 \to 7$、$3 \to 4$、$5 \to 6$、$8 \to 3$、$9 \to 8$ 等。总体来看，该 CNN 已能较好地识别 MNIST 数字，但在局部形态相近、闭合程度不清晰或笔画倾斜明显的样本上仍容易混淆。

6. 实验小结

本实验完成了 Topic 7 Part 1 实验 7-1 的主要要求：实现卷积层、平均池化层与 MNIST 数据加载函数，搭建了 28×28 输入、20 个 9×9 卷积核、2×2 平均池化、100 节点隐藏层和 10 类 Softmax 输出层组成的 CNN，并使用 mini-batch momentum 规则训练 10 个 epoch。在 2000 张训练图像与 300 张测试图像设置下，最终测试准确率达到 94.33%。训练/测试错误率曲线、卷积核、feature map 和错分样本可视化共同说明：卷积层能够提取数字笔画的局部特征，池化层降低了空间维度，全连接层和 Softmax 层可以在这些特征基础上完成分类。